Amal9137



WinOS-EFI

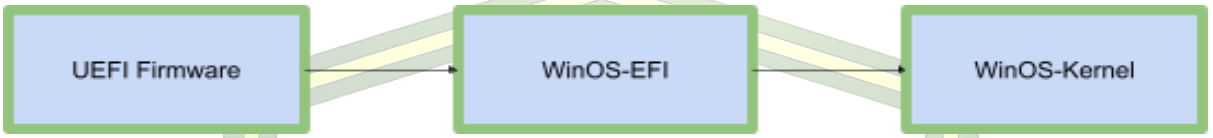
TABLE OF CONTENTS

Why WinOS-EFI? **2**
What does it do? **2**
BootConfig 3
ExitBootServices() 4
Failure Detection 4

Why WinOS-EFI?

WinOS-EFI is essential for many modern devices, such as desktop computers and laptops, that use the **UEFI specification**. It is designed to act as the bridge between the machine's firmware and WinOS itself.

When a machine with WinOS installed starts, **WinOS-EFI** is the first WinOS component to launch. As an **EFI Application**, it runs directly within the UEFI environment before the **WinOS System (WinOS-Kernel)** is started.



What does it do?

On UEFI-based systems, WinOS-Kernel requires a UEFI application to prepare the boot environment and provide the information and resources required for execution. This is where **WinOS-EFI** comes into play.

WinOS-EFI is responsible for loading an ***index.wfs*** file *(a WinOS-Dev derivative that is planned to be deprecated)*, which contains the locations of essential WinOS components such as the kernel, logo, and other boot resources. After locating the required components, **WinOS-EFI** obtains the system's **memory map** and builds the **BootConfig** structure. **BootConfig** contains information obtained from the **UEFI environment**, such as the current display resolution and framebuffer information, which **WinOS-Kernel** *requires* when it begins execution.

WinOS-EFI then attempts to **exit UEFI Boot Services**, transitioning control from the UEFI environment to WinOS-Kernel. Before loading critical components, **WinOS-EFI performs several integrity and validity checks**, including **DOS signature** and **PE header** checks. These checks help ensure that the files being loaded are **valid executable images** rather than **arbitrary** or **malformed files**.

In the future, **WinOS** is planned to include a **WinOS System Integrity Check**, which will provide a **stronger verification mechanism** for **critical system binaries**, including the **kernel**, by *verifying their contents before execution*.

BootConfig

The **BootConfig** structure contains the following data:

- **BaseAddress** — The memory address of the framebuffer containing the pixels currently displayed on the screen.
- **BackBuffer** — The memory address of the back buffer. It is the same size as the framebuffer and is used to prepare frames before displaying them, providing smoother rendering before dedicated graphics drivers are available.
- **Width** — The horizontal resolution of the display, measured in pixels.
- **Height** — The vertical resolution of the display, measured in pixels.
- **PixelsPerScanLine** — The number of pixels allocated for each horizontal scan line of the framebuffer. This may be greater than `Width` because the framebuffer can contain additional padding or unused pixels at the end of each scan line.
- **Mouse** — Properties and information related to the cursor. **Deprecated; planned for removal.**
- **Font** — A basic **PSF (PC Screen Font)** font used for early text rendering. **Deprecated; planned for removal.**
- **Assets** — The file index loaded from `index.wfs`. **Deprecated; planned for removal.**
- **MemMap** — The memory address of the UEFI system memory map.
- **MemMapSize** — The total size of the UEFI memory map.
- **MemMapDescriptorSize** — The size of each individual memory-map descriptor.

BootConfig is constructed by **WinOS-EFI** and provided to **WinOS-Kernel** when *control is transferred*.

```c
// The ultimate payload handed to the Kernel
typedef struct {
    uint32_t* BaseAddress;       // The actual screen pixels
    uint32_t* BackBuffer;        // The hidden drawing canvas
    uint32_t  Width;
    uint32_t  Height;
    uint32_t  PixelsPerScanLine;

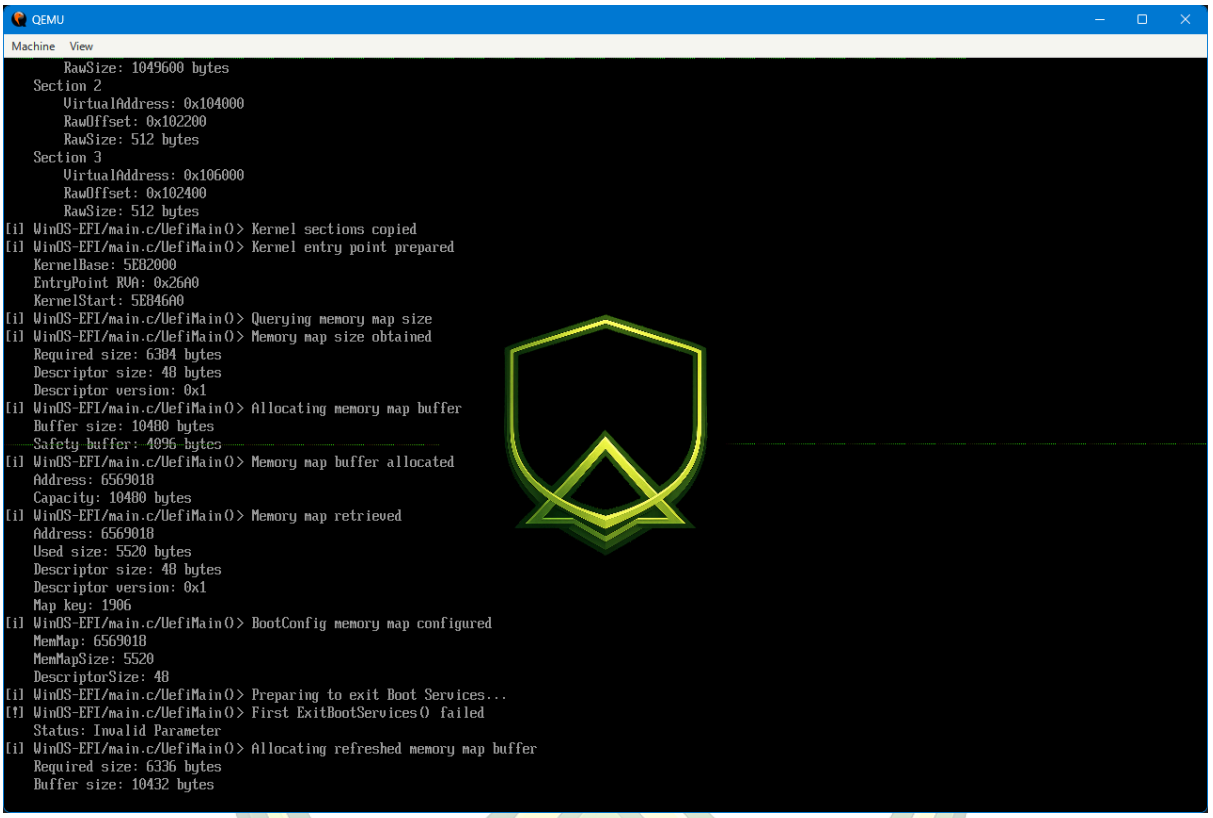
    MOUSE_STATE* Mouse;
    PSF1_HEADER* Font;
    ASSET_TABLE* Assets;         // The dynamic Index Table

    // Pure pointers to avoid breaking compiler cross-compatibility
    void* MemMap;
    uint64_t  MemMapSize;
    uint64_t  MemMapDescriptorSize;
} BOOT_CONFIG;
```

Current BOOT_CONFIG as of 26y9m5

ExitBootServices()

After **ExitBootServices()** succeeds, *UEFI Boot Services are no longer available to WinOS, and control is transferred to WinOS-Kernel*.



The WinOS Logo is a temporary render by WinOS-Kernel. This is to be removed after implementation of the WinOS System.

Failure Detection

WinOS-EFI *dynamically* reports detected *failures and successful boot stages through its diagnostic output*, making **boot-time debugging easier**.